

大模型网关 LiteLLM 路由策略

参考 LiteLLM 官方文档: <https://docs.litellm.ai/docs/routing>

背景说明

LiteLLM 的 Router 策略机制非常丰富, 也很适合真实业务场景。它不只是“随机挑一个模型供应商”这么简单, 而是把负载均衡、限流感知、低延迟优先、低成本优先、失败重试、fallback 等能力都放在同一个路由层里。

这套机制最实用的地方在于: 业务代码只需要请求一个统一的 `model_name`, 背后具体走哪个供应商、哪个区域、哪个部署, 可以交给 LiteLLM 根据策略自动选择。这样一来, 后续想加供应商、调流量比例、切备用模型、做限流保护, 都可以优先在网关配置里完成, 而不需要频繁改业务代码。

在生产环境里, LiteLLM Router 常见可以解决这些问题:

- 多个部署之间的负载均衡
- 根据 RPM/TPM 或权重分配请求
- 请求前过滤可能超限的部署
- 根据延迟、成本、并发忙碌程度选择部署
- 某个部署失败后自动重试、冷却或 fallback
- 不同模型组使用不同路由策略

LiteLLM Router 就是用来解决这些问题的。它会把相同 `model_name` 的多个部署视为同一个模型组, 业务请求只需要调用这个模型组名, Router 再根据配置选择具体供应商或部署。

如果还不会使用 LiteLLM, 可以先看我的主页: [大模型网关 LiteLLM 零基础快速上手教程](#)。

下面的示例会用 `deepseek-v4-flash` 作为业务模型名, 假设它背后接了三个供应商:

1. [DeepSeek 官方](#)
2. [阿里云百炼](#)
3. [硅基流动](#)

最基础的负载均衡

只要多个配置使用相同的 `model_name`，LiteLLM 就会把它们放进同一个模型组。业务侧请求 `model: deepseek-v4-flash`，网关会在这个模型组里选择一个实际部署。

```
model_list:
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-v4-flash
    api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
    api_key: os.environ/DASHSCOPE_API_KEY

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-ai/DeepSeek-V4-Flash
    api_base: https://api.siliconflow.cn/v1
    api_key: os.environ/SILICONFLOW_API_KEY

router_settings:
  routing_strategy: simple-shuffle

general_settings:
  master_key: sk-1234
```

路由策略总览

LiteLLM Router 支持以下路由策略：

策略	配置项	适合场景
加权随机，默认推荐	<code>routing_strategy: simple-shuffle</code>	生产默认选择，性能最好，可按 <code>rpm</code> 、 <code>tpm</code> 、 <code>weight</code> 分配
部署优先级	<code>order</code>	主供应商优先，主供应商不可用才走备用供应商
限流感知 v2	<code>routing_strategy: usage-based-routing-v2</code>	异步统计 RPM/TPM，过滤超限并选当前分钟 TPM 最低的部署
延迟优先	<code>routing_strategy: latency-based-routing</code>	多区域、多供应商质量不同，希望优先选择响应快的部署
最少忙碌	<code>routing_strategy: least-busy</code>	请求耗时差异大，希望选择当前并发请求最少的部署
成本优先	<code>routing_strategy: cost-based-routing</code>	多供应商价格差异大，希望优先选择便宜部署

官方建议生产环境优先使用 `simple-shuffle` 。基于使用量的策略会增加 Redis 读写和统计开销，高并发场景要谨慎。

策略一：加权随机 `simple-shuffle`

`simple-shuffle` 是默认且推荐的策略。它会根据部署上配置的 `rpm` 、`tpm` 或 `weight` 来分配请求；如果都没有配置，则随机选择一个健康部署。

按 `weight` 分配

控制大概比例，可以使用 `weight` 。

```
model_list:
  - model_name: deepseek-v4-flash
    litellm_params:
      model: deepseek/deepseek-v4-flash
      api_base: https://api.deepseek.com
```

```

    api_key: os.environ/DEEPSEEK_API_KEY
    weight: 7

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-v4-flash
    api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
    api_key: os.environ/DASHSCOPE_API_KEY
    weight: 2

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-ai/DeepSeek-V4-Flash
    api_base: https://api.siliconflow.cn/v1
    api_key: os.environ/SILICONFLOW_API_KEY
    weight: 1

router_settings:
  routing_strategy: simple-shuffle

```

这个配置大致表示 70% 请求走 DeepSeek 官方，20% 走百炼，10% 走硅基流动。

按 RPM/TPM 分配

适合不同供应商额度不同的情况，比如 DeepSeek 官方额度更高，硅基流动额度较低。

```

model_list:
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY
    rpm: 900
    tpm: 900000

```

```
- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-v4-flash
    api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
    api_key: os.environ/DASHSCOPE_API_KEY
    rpm: 1
    tpm: 100

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-ai/DeepSeek-V4-Flash
    api_base: https://api.siliconflow.cn/v1
    api_key: os.environ/SILICONFLOW_API_KEY
    rpm: 1
    tpm: 100

router_settings:
  routing_strategy: simple-shuffle
```

这里不是简单的三家平均分，而是会按各自可承载的请求量进行倾斜。

策略二：部署优先级 order

给同一个模型组里的部署配置 `order`，可以控制主备优先级。数值越小，优先级越高；同一优先级内，再交给当前 `routing_strategy` 选择。

```
model_list:
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY
    order: 1
```

```

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-v4-flash
    api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
    api_key: os.environ/DASHSCOPE_API_KEY
    order: 2

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-ai/DeepSeek-V4-Flash
    api_base: https://api.siliconflow.cn/v1
    api_key: os.environ/SILICONFLOW_API_KEY
    order: 3

router_settings:
  routing_strategy: simple-shuffle

```

上面的配置表示：默认优先走 DeepSeek 官方；官方不可用（失败、冷却、限流等）时，再依次尝试百炼、硅基流动。

`order` 可以和任意 `routing_strategy` 组合使用。例如主备模式下，`order: 1` 的部署始终优先；只有它不可用时，才会在 `order: 2` 的部署里继续按 `simple-shuffle`、`latency-based-routing` 等策略选择。

工作原理

源码里 `order` 的处理发生在策略选择之前。Router 会先找到当前模型组下所有健康部署，然后调用 `litellm.utils._get_order_filtered_deployments()` 做过滤：

- 如果没有指定 `_target_order`，只保留最小 `order` 的部署，比如只保留 `order: 1`。
- 如果当前请求是 `order fallback` 过程，会通过 `_target_order` 指定下一档优先级，比如从 `order: 1` 切到 `order: 2`。
- 没有配置 `order` 的部署不会优先于显式配置了较小 `order` 的部署。

这里的 `_target_order` 不是用户配置项，而是 Router 在内部 fallback 时临时塞进请求参数里的标记。比如第一次请求 `deepseek-v4-flash` 时，候选池会先被过滤成 `order: 1` 的部署；如果这一档请求失败，并且同一个模型组里还存在 `order: 2`、`order: 3`，Router 会在构造 fallback 列表时生成类似 `{"model": "deepseek-v4-flash", "_target_order": 2}` 的内部重试目标。下一次进入 `get_available_deployment` 时，`_get_order_filtered_deployments()` 看到 `_target_order: 2`，就不会再选最低的 `order: 1`，而是只保留 `order: 2` 的部署。

所以 `order` 的 fallback 不是直接换到另一个 `model_name`，而是先在同一模型组里按优先级档位往后尝试。只有这些 `order` 档位都试完，或者没有可用部署时，才会继续进入你显式配置的跨模型组 `fallbacks`。

过滤之后，剩下的部署才会进入当前 `routing_strategy`。所以如果 `order: 1` 下面有两个部署，并且策略是 `simple-shuffle`，这两个部署会继续按 `simple-shuffle` 负载均衡；只有这一档都不可用、失败或被冷却时，Router 的 fallback 流程才会把下一档 `order` 加入尝试列表。

适合的场景：

- 主供应商优先，备用供应商兜底
- 官方 API 额度充足，第三方只做灾备
- 希望控制成本，优先走便宜或自有渠道

策略三：限流感知 v2

官方文档里把 Rate-Limit Aware v2 作为新的异步实现来介绍，对应策略是 `usage-based-routing-v2`。它会根据部署的 `rpm`、`tpm` 做使用量统计，并过滤已经超限的部署。

生产环境建议配合 Redis 使用，因为多个 LiteLLM 实例需要共享限流计数。

```
model_list:
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY
    rpm: 900
    tpm: 900000
```

```
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY2
    rpm: 900
    tpm: 900000

router_settings:
  routing_strategy: usage-based-routing-v2
  enable_pre_call_check: true
  redis_host: os.environ/REDIS_HOST
  redis_password: os.environ/REDIS_PASSWORD
  redis_port: os.environ/REDIS_PORT
```

适合的场景：

- 有明确 RPM/TPM 限额
- 不希望请求已经发到供应商之后才收到限流错误
- 希望尽量把每个供应商的额度用得更均匀
- 多个 LiteLLM Proxy 实例共同承接流量

如果你更看重极致吞吐，官方仍然建议优先使用 `simple-shuffle`，再配合 `enable_pre_call_check` 做请求前检查。

工作原理

`usage-based-routing-v2` 的实现主要在 `litellm/router_strategy/lowest_tpm_rpm_v2.py`。它的核心思想是：按部署维度记录当前分钟的 TPM/RPM 使用量，路由时过滤掉预计会超限的部署，再从剩余部署中选择当前 TPM 最低的一个。

一次异步请求大致是这样走的：

1. Router 先拿到健康部署列表。

2. v2 selector 为每个候选部署拼出两个 key: `{deployment_id}:{backend_model}:tpm:{HH-MM}` 和 `{deployment_id}:{backend_model}:rpm:{HH-MM}`。
3. 通过 `async_batch_get_cache()` 一次批量读取所有候选部署的 TPM/RPM 当前值。使用 Redis 时, 多个 LiteLLM 实例可以共享这些计数。
4. 根据请求输入预估 `input_tokens`。
5. 过滤掉 `current_tpm + input_tokens > tpm` 或 `current_rpm + 1 >= rpm` 的部署。
6. 在剩余部署中找当前 TPM 最低的部署; 如果多个部署 TPM 一样低, 就随机选一个, 避免固定打到同一个部署。

v2 还有一个很关键的 pre-call check: 真正发请求前会先递增 RPM 计数, 如果递增后发现超过配置的 `rpm`, 会直接抛 `RateLimitError`, 请求不会再发给供应商。成功返回之后, 再根据真实 `total_tokens` 增加 TPM 计数。

策略四: 延迟优先 latency-based-routing

`latency-based-routing` 会记录各个部署的响应耗时, 并优先选择平均延迟最低的部署。

```
model_list:
  - model_name: deepseek-v4-flash
    litellm_params:
      model: deepseek/deepseek-v4-flash
      api_base: https://api.deepseek.com
      api_key: os.environ/DEEPSEEK_API_KEY

  - model_name: deepseek-v4-flash
    litellm_params:
      model: openai/deepseek-v4-flash
      api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
      api_key: os.environ/DASHSCOPE_API_KEY

router_settings:
  routing_strategy: latency-based-routing
  enable_pre_call_check: true
```

可以通过 `routing_strategy_args.ttl` 控制统计窗口。例如只参考最近 10 秒的延迟：

```
router_settings:
  routing_strategy: latency-based-routing
  routing_strategy_args:
    ttl: 10
```

也可以配置 `lowest_latency_buffer`，不要把所有请求都压到当前最快的那个部署上。比如最快节点 100ms，buffer 为 0.5 时，150ms 内的节点也会进入候选范围。

```
router_settings:
  routing_strategy: latency-based-routing
  routing_strategy_args:
    lowest_latency_buffer: 0.5
```

适合的场景：

- 多个供应商网络质量差异明显
- 多区域部署，离用户近的节点更快
- 更关注响应速度，而不是严格按额度比例分配

工作原理

实现位于 `litellm/router_strategy/lowest_latency.py`。这个策略不是实时探测所有供应商，而是基于历史请求日志做选择。

每次请求成功后，handler 会把部署的延迟写入 `{model_group}_map` 缓存。缓存结构里每个部署会有：

- `latency`：最近若干次非流式或完整请求的延迟样本，默认最多保留 10 个。
- `time_to_first_token`：流式请求的首 token 时间样本。
- 当前分钟的 `tpm / rpm` 统计。

下一次路由时，它会先过滤掉预计超过 TPM/RPM 的部署，然后计算每个候选部署的平均延迟。普通请求使用 `latency`；如果请求是 `stream: true` 且该部署有 `time_to_first_token` 样本，就用 TTFT 平均值，更贴近流式体验。

选中逻辑不是永远拿绝对最低延迟的部署。源码会先按平均延迟排序，找到最低延迟，再根据 `lowest_latency_buffer` 计算一个可接受范围：最低延迟 + 最低延迟 * `buffer`。所有落在这个范围内的部署都会进入最终候选，再随机选一个。这样可以避免所有请求都压到当前最快的单个部署上。

如果某个部署发生超时，异步失败日志会给这个部署写入一个很大的延迟惩罚值，后续一段时间内它就不容易被选中。

策略五：最少忙碌 `least-busy`

`least-busy` 会选择当前正在处理请求数量最少的部署。

```
model_list:
  - model_name: deepseek-v4-flash
    litellm_params:
      model: deepseek/deepseek-v4-flash
      api_base: https://api.deepseek.com
      api_key: os.environ/DEEPSEEK_API_KEY

  - model_name: deepseek-v4-flash
    litellm_params:
      model: openai/deepseek-v4-flash
      api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
      api_key: os.environ/DASHSCOPE_API_KEY

router_settings:
  routing_strategy: least-busy
```

适合的场景：

- 请求耗时差异很大
- 有些请求会长时间流式输出
- 希望避免某个部署堆积太多并发请求

工作原理

实现位于 `litellm/router_strategy/least_busy.py`。它依赖 LiteLLM 的 callback 机制维护一个“正在飞行中的请求数”。

请求即将发出时，`log_pre_api_call()` 会根据 `model_group` 和部署 `model_info.id`，把 `{model_group}_request_count` 里的对应部署计数加 1。请求成功或失败后，`log_success_event()` / `log_failure_event()` 再把这个计数减 1。异步路径也是同样逻辑，只是使用 `async cache API`。

路由时，selector 会读取 `{model_group}_request_count`，没有出现过的健康部署默认计数为 0，然后选择计数最小的部署。如果缓存里找不到匹配部署，或者没有可用计数，就退化为随机选择一个健康部署。

这个策略关心的是“当前并发中的请求数量”，不是历史平均延迟，也不是 token 使用量。它特别适合请求耗时差异大、流式输出长、单个部署容易被长连接占住的场景。

策略六：成本优先 **cost-based-routing**

`cost-based-routing` 会从健康部署中选择成本最低的部署。LiteLLM 会优先使用内置模型价格表；如果模型不在价格表里，可以手动配置输入和输出 token 单价。

```
model_list:
- model_name: deepseek-v4-flash
  litellm_params:
    model: deepseek/deepseek-v4-flash
    api_base: https://api.deepseek.com
    api_key: os.environ/DEEPSEEK_API_KEY
    input_cost_per_token: 0.00000014
    output_cost_per_token: 0.00000028

- model_name: deepseek-v4-flash
  litellm_params:
    model: openai/deepseek-v4-flash
    api_base: https://dashscope.aliyuncs.com/compatible-mode/v1
    api_key: os.environ/DASHSCOPE_API_KEY
    input_cost_per_token: 0.00000027
```

```
output_cost_per_token: 0.00000110

router_settings:
  routing_strategy: cost-based-routing
```

适合的场景：

- 同一个模型或相近模型在不同供应商价格差异明显
- 离线任务、批处理任务，对延迟不敏感
- 希望默认走低成本供应商，高成本供应商只作为兜底

工作原理

实现位于 `litellm/router_strategy/lowest_cost.py`。当前源码里的成本策略走异步 selector，`Router._select_deployment_sync()` 里明确没有接入 `cost-based-routing` 的同步选择逻辑，所以在 Proxy/异步调用路径里使用最稳。

路由时，它会遍历健康部署，并为每个部署计算一个单价：

1. 优先读取部署 `litellm_params.input_cost_per_token` 和 `output_cost_per_token`。
2. 如果没有手动配置，就从 `litellm.model_cost` 内置价格表里按后端模型名读取。
3. 如果价格表也没有，源码给输入和输出价格各自使用默认值 `5.0`，也就是总成本会变成很高的默认值，避免未知价格模型被误判成便宜。

然后它同样会用当前分钟的 `tpm / rpm` 统计过滤超限部署，剩余部署按 `input_cost_per_token + output_cost_per_token` 从低到高排序，选择最便宜的一个。

所以它并不会根据本次请求的输入/输出比例计算精确账单，而是用“输入单价 + 输出单价”作为部署排序指标。对于同一类请求来说已经足够稳定；如果你的请求输入很长但输出很短，或者输出特别长，最好把候选模型的价格差异和业务 token 结构一起考虑。

分模型配置不同策略：Routing Groups

如果一个 LiteLLM 网关里同时代理多个模型，可以给不同模型组配置不同策略。例如：

- `deepseek-v4-flash`：默认加权随机
- `gpt-4o`：延迟优先

- `batch-model` : 成本优先

`model_list:`

- `model_name: deepseek-v4-flash`
`litellm_params:`
 - `model: deepseek/deepseek-v4-flash`
 - `api_base: https://api.deepseek.com`
 - `api_key: os.environ/DEEPSEEK_API_KEY`
- `model_name: gpt-4o`
`litellm_params:`
 - `model: openai/gpt-4o`
 - `api_key: os.environ/OPENAI_API_KEY`
- `model_name: gpt-4o`
`litellm_params:`
 - `model: azure/gpt-4o`
 - `api_base: os.environ/AZURE_API_BASE`
 - `api_key: os.environ/AZURE_API_KEY`
 - `api_version: "2024-08-01-preview"`
- `model_name: batch-model`
`litellm_params:`
 - `model: openai/gpt-4o-mini`
 - `api_key: os.environ/OPENAI_API_KEY`

`router_settings:`

- `routing_strategy: simple-shuffle`
- `routing_groups:`
 - `group_name: latency-sensitive`
 - `models: [gpt-4o]`
 - `routing_strategy: latency-based-routing`
 - `routing_strategy_args:`
 - `ttr: 3600`

```
- group_name: cost-sensitive
  models: [batch-model]
  routing_strategy: cost-based-routing
```

注意事项:

- 每个 `model_name` 最多只能属于一个 group
- 没有命中任何 group 的模型会使用顶层 `router_settings.routing_strategy`
- `default` 是保留 group 名, 不要自己使用

也可以把批处理模型单独配置成 `usage-based-routing-v2` :

```
router_settings:
  routing_strategy: simple-shuffle
  routing_groups:
    - group_name: batch
      models: [batch-model]
      routing_strategy: usage-based-routing-v2
      routing_strategy_args:
        ttl: 60
```

和路由强相关的可靠性配置

路由策略只决定“优先选谁”, 生产环境还需要配合重试、超时、冷却和 fallback。

重试

```
router_settings:
  num_retries: 2
  retry_after: 5
```

超时

```
router_settings:  
  timeout: 30
```

失败冷却

当某个部署连续失败，LiteLLM 可以把它临时移出候选池，过一段时间再恢复。

```
router_settings:  
  cooldown_time: 60  
  allowed_fails: 3
```

fallback

当一个模型组完全不可用时，可以自动切到另一个模型组。

```
model_list:  
  - model_name: deepseek-v4-flash  
    litellm_params:  
      model: deepseek/deepseek-v4-flash  
      api_base: https://api.deepseek.com  
      api_key: os.environ/DEEPSEEK_API_KEY  
  
  - model_name: deepseek-backup  
    litellm_params:  
      model: openai/deepseek-v4-flash  
      api_base: https://dashscope.aliyuncs.com/compatible-mode/v1  
      api_key: os.environ/DASHSCOPE_API_KEY  
  
router_settings:  
  fallbacks:  
    - deepseek-v4-flash: [deepseek-backup]
```

weighted failover

如果同一个 `model_name` 下面有多个部署，默认失败后可能进入跨模型组 `fallback`。开启 `enable_weighted_failover` 后，Router 会先在同一个模型组里排除刚失败的部署，再按原有 `weight`、`rpm`、`tpm` 重新选择其他部署；同组都失败后，才进入跨模型组 `fallback`。

```
router_settings:
  routing_strategy: simple-shuffle
  enable_weighted_failover: true
  max_fallbacks: 5
```

这个配置只适用于 `simple-shuffle`，很适合多个区域部署的是同一个模型，只希望某个区域失败时先切到同模型的其他区域。

选型建议

业务目标	推荐策略
生产默认、吞吐优先	<code>simple-shuffle</code>
按供应商额度分配	<code>simple-shuffle</code> + <code>rpm</code> / <code>tpm</code>
按固定比例分配	<code>simple-shuffle</code> + <code>weight</code>
主备 / 主供应商优先	<code>order</code>
请求前过滤超限部署 / 额度均衡	<code>usage-based-routing-v2</code> + Redis, 或 <code>simple-shuffle</code> + <code>enable_pre_call_check</code>
追求最低延迟	<code>latency-based-routing</code>
长请求、流式请求多	<code>least-busy</code>
批处理、省钱优先	<code>cost-based-routing</code>

如果你刚开始搭建大模型网关，建议先使用：

```
router_settings:
  routing_strategy: simple-shuffle
```

```
enable_pre_call_check: true  
num_retries: 2  
timeout: 30  
cooldown_time: 60
```

然后根据线上日志逐步增加 rpm 、 tpm 、 weight 、 fallback 和 Redis。